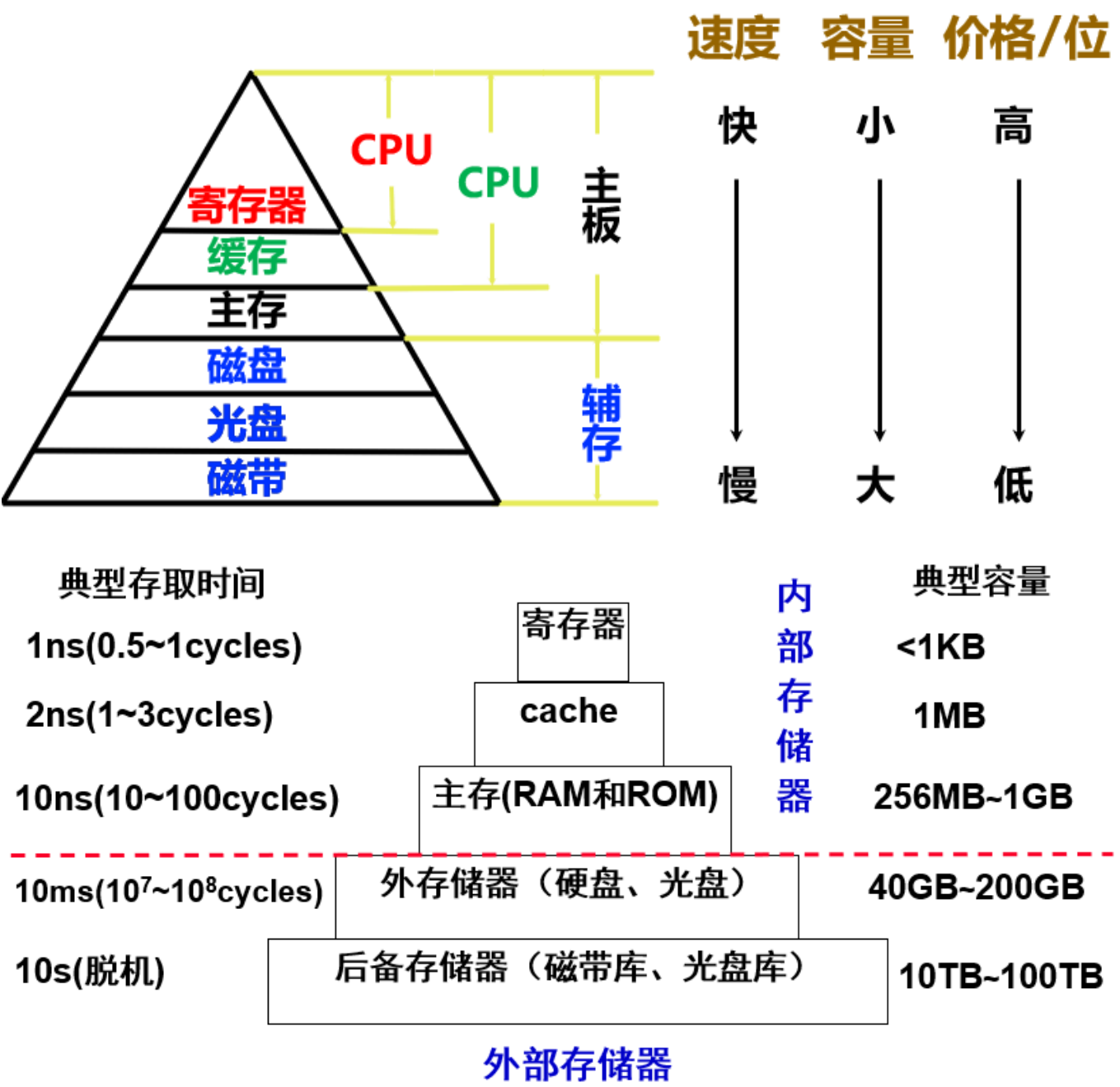


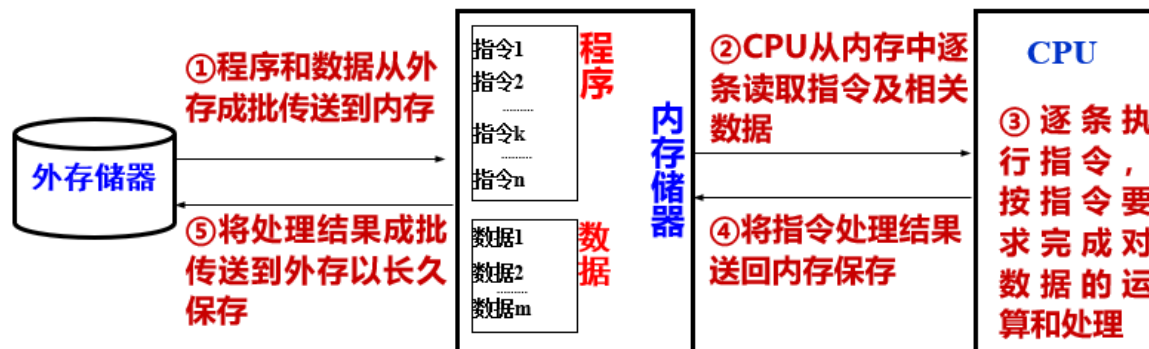
6 - 层次结构存储系统

作者：Klein White
时间：2025-04-21
概要：

存储器的分类



- 寄存器：封装在 CPU ，存放当前执行的指令和数据，数量少
- 高速缓存：位于 CPU 内部或附近，存放当前执行的局部程序和数据
- 内存/主存：位于 CPU 之外，用于存放已经被启动的程序和数据
- 外存：位于主机之外，存放暂不运行的程序和文件等

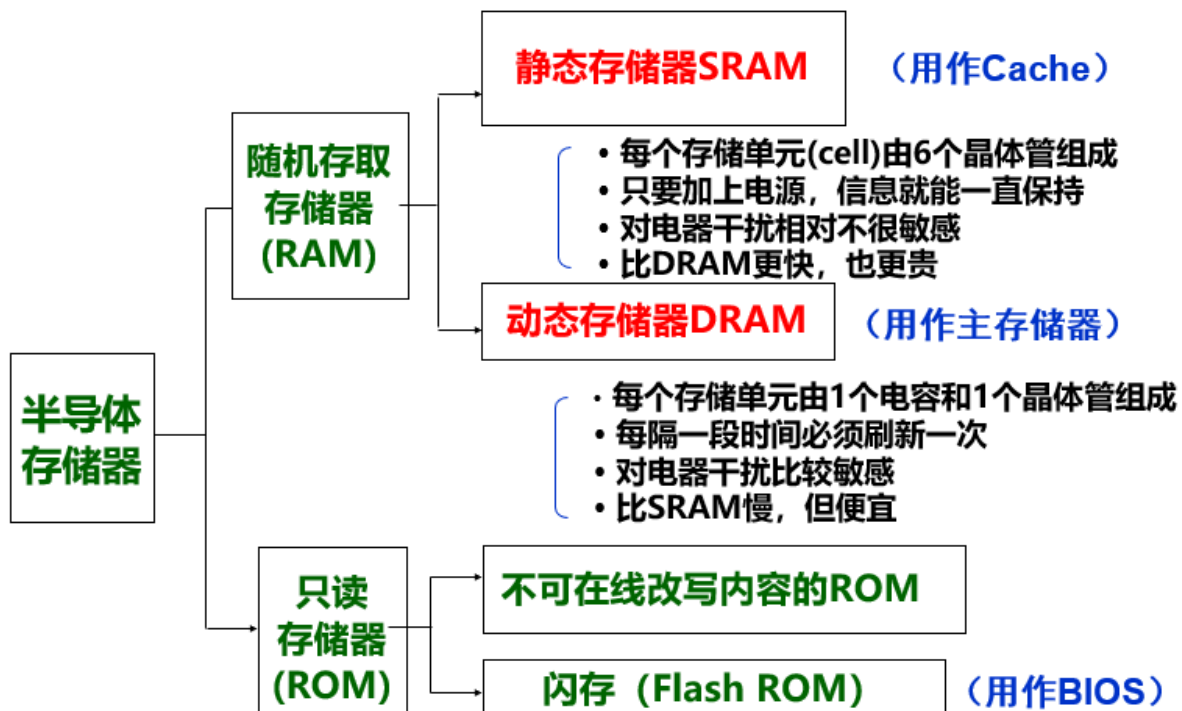


✓ 外存储器 (简称外存或辅存)

- 存取速度慢
- 成本低、容量很大
- 不与CPU直接连接，先传送到内存，然后才能被CPU使用。
- 属于**非易失性**存储器，用于长久存放系统中几乎所有的信息

✓ 内存存储器 (简称内存或主存)

- 存取速度快
- 成本高、容量相对较小
- 直接与CPU连接，CPU对内存中可直接进行读、写操作
- 属于**易失性**存储器(volatile)，用于临时存放正在运行的程序和数据



SRAM 通常用于作为高速缓存， DRAM 则大多用于主存（内存条）

存储芯片的扩容

字扩展

比如用两个 $1K \times 8$ 位的DRAM芯片设计 $2K \times 8$ 位的DRAM存储器，这需要 11 根地址线和8条数据线

位扩展

比如用两个 $1K \times 4$ 位的SRAM芯片，设计一个存储容量为 $1K \times 8$ 位的SRAM存储器，地址线还是10根，但是需要8跟数据线

字-位扩展

比如用八个 $1K \times 4$ 位 存储芯片组成 $4K \times 8$ 位 的存储器

高速缓存

为什么引入？

程序访问的局限性（时间局限性、空间局限性）：大量程序结果表明，较短时间间隔内，程序产生的地址和访问的数据往往集中在一个很小的范围

程序具有访问局部性特征的原因：

- 指令：指令按序存放，地址连续，循环程序段或子程序段重复执行
- 数据：连续存放，数组元素重复、按序访问

所以在CPU和主存之间设置一个快速小容量的存储器，其中总是存放最活跃（被频繁访问）的程序和数据，由于程序访问的局部性特征，大多数情况下，CPU能直接从这个高速缓存中取得指令和数据，而不必访问主存 --> Cache 高速缓存

Cache 是一种小容量的高速缓冲存储器，由 SRAM 组成，其速度几乎和CPU一样快

Cache的内容是**主存储器中部分内容的映象**（也就是内存中一部分数据会copy到缓存里面）

当CPU需要从内存读(写)数据或指令时，先检查Cache，若有（命中），就直接从Cache中读取，

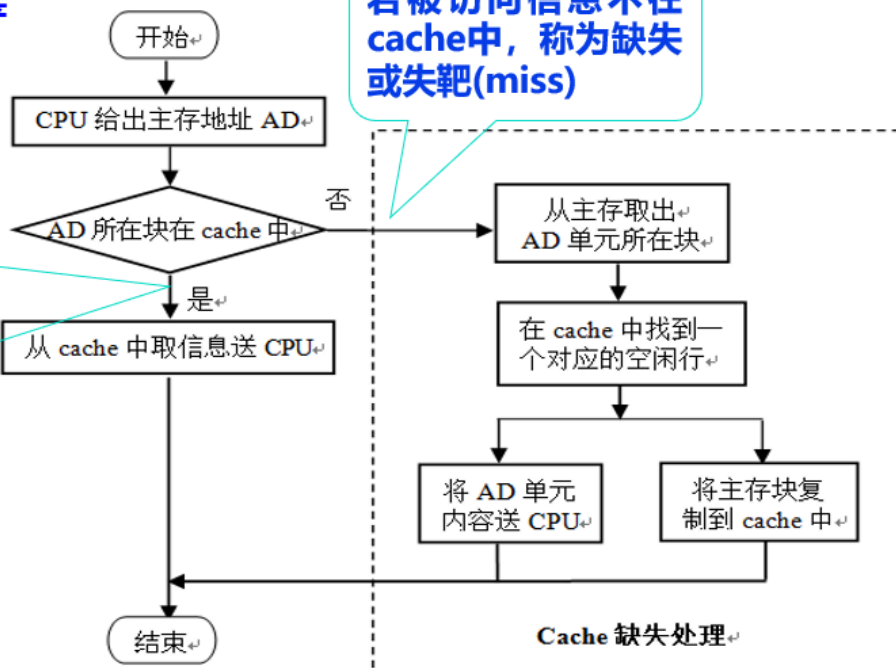
而不用访问主存储器。

问题：什么情况下，CPU产生访存要求？

执行指令时！

若被访问信息在cache中，称为命中(hit)

若被访问信息不在cache中，称为缺失或失靶(miss)



主存中，数据分成若干个大小相同的块，称为主存块。

Cache 里面也分成相同大小的块，叫做 Cache 行或 槽

Cache 映射

把主存中的一些主存块复制到 Cache 的时候，要放在 Cache 的那一个 Cache 行？

需要考虑到Cache行比主存块少，所以可能多个主存块映射到一个Cache行

将主存块和Cache行按照以下三种方式进行映射：

- 直接映射：每个主存块映射到Cache的固定行
- 全相联映射：每个主存块映射到Cache的任一行
- 组相联映射：每个主存块映射到Cache固定组中任一行

直接映射（模映射）

把主存的每一块映射到一个固定的Cache行

映射关系为：**Cache行号=主存块号 mod Cache行数**

例如： $4=100 \bmod 16$ 主存第100块应映射到Cache的第4行中

特点：

- 容易实现，命中时间短
- 无需考虑淘汰问题
- 不够灵活，Cache 的存储空间利用不充分，命中率低

这样的情况下，假设主存地址为20位，那么访问时候的地址结构和过程如下：
根据地址中间4位找到Cache相应的槽，然后取出该槽的标志，与地址中高7位进行比较。 若相等，则说明该主存单元所在的块在Cache中

假定数据在主存和Cache间的传送单位为512B。

Cache大小：
 $2^{13}B = 8KB = 16行 \times 512B/行$

主存大小：
 $2^{20}B = 1024KB = 2048块 \times 512B/块$

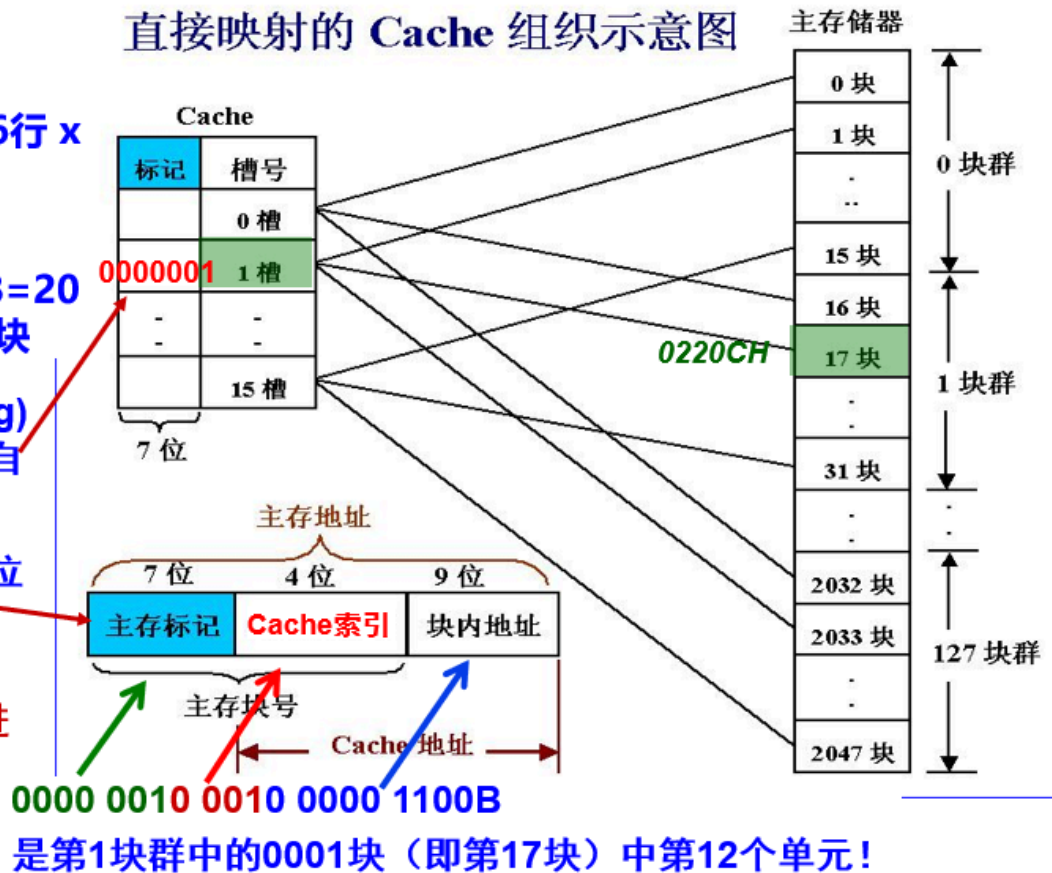
Cache标记(tag)
指出对应行取自哪个主存块群

指出对应地址位于哪个块群

例：如何对0220CH单元进行访问？

直接映射Cache组织示意图

直接映射的 Cache 组织示意图



全相联映射

每个主存块可装到Cache任一行中。

所以在地址中，没有了 cache 索引字段。

从 Cache 查找数据的时候，首先根据地址的高11为来对比标记，若能找到相等的槽，则说明要

访问的单元在该槽中。再根据后9位字号找到相应的字取到CPU中。

假定数据在主存和Cache间的传送单位为512B。

Cache大小：
 $2^{13}B=8KB=16行 \times 512B/行$

主存大小：
 $2^{20}B=1024KB=2048块 \times 512B/块$

Cache标记 (tag) 指出对应行取自哪个主存块

主存tag指出对应地址位于哪个主存块

如何对01E0CH单元进行访问？

0000 0001 1110 0000 1100B
是第15块中的第12个单元！

全相联映射Cache组织示意图

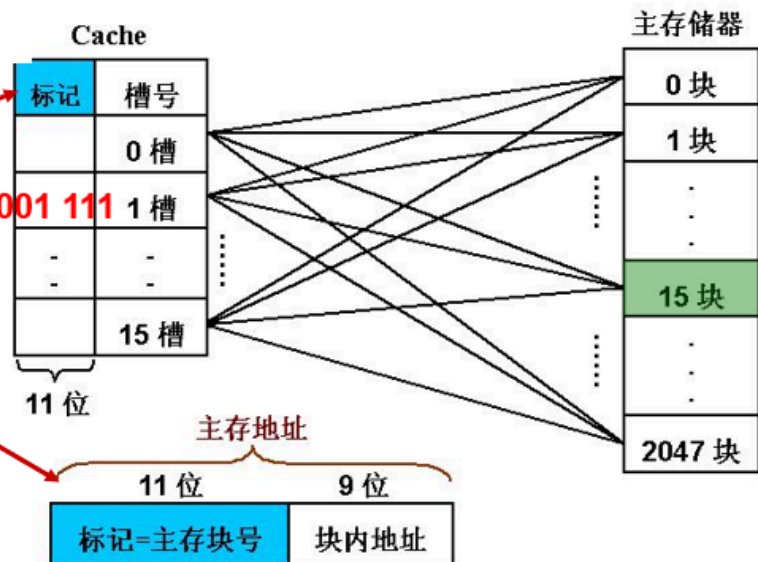
按内容访问，是相联存取方式！

每个主存块可装到Cache任一行中。

如何实现按内容访问？

直接比较！

全相联映射的 Cache 组织示意图



为何地址中没有cache索引字段？
因为可映射到任意一个cache行中！

组相联映射

结合直接映射和全相联映射的特点

将Cache所有行分组，把主存块映射到Cache固定组的任一行中。也即：组间模映射、组内全映射。

映射关系为：Cache组号=主存块号 mod Cache组数

相邻的主存块被分到不同的 Cache 组内，而 Cache 组内的映射是任意的

特点：

- 结合直接映射和全相联映射的优点。当Cache组数为1时，变为全相联映射；当每组只有一个槽时，变为直接映射
- 每组2或4行（称为2-路或4-路组相联）较常用。通常每组4行以上很少用

查找 Cache 的过程：

根据中间3位的内容找到对应的Cache组，再将前8位同时与该组中各槽的标志位进行比较。若能

找到相等的槽，则说明要访问的单元在该槽中。

假定数据在主存和
Cache间的传送单
位为512B。

Cache大小：
 $2^{13}B=8KB=16行 \times$
512B/ 行

主存大小：
 $2^{20}B=1024KB=20$
48块 $\times$ 512B/ 块

指出对应行取自哪
个主存组群

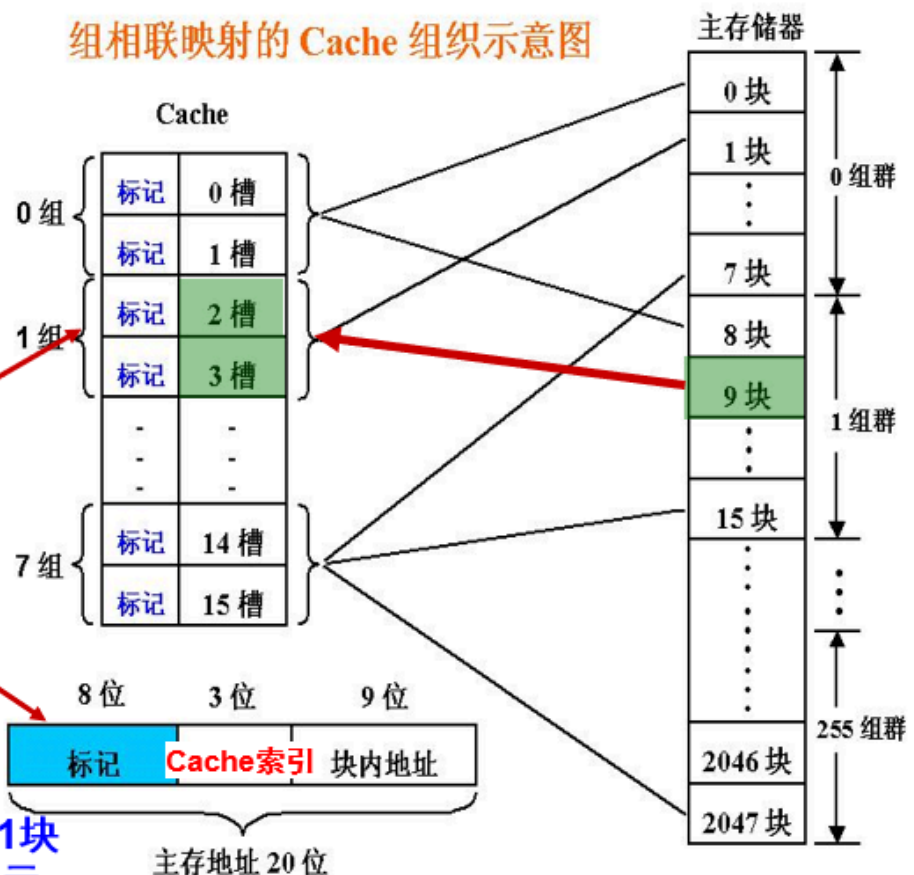
指出对应地址位于
哪个主存组群中

例：如何对0120CH单
元进行访问？

0000 0001 0010 0000

1100B是第1组群中的001块
(即第9块) 中第12个单元。
所以，映射到第一组中。

组相联映射的 Cache 组织示意图



将主存地址标记和对应Cache组中
每个Cache标记进行比较！

Cache的一致性问题

Cache 的内容是主存块的副本，当对Cache中的内容进行更新时，就存在Cache和主存如何保持一致的问题。

可能出现 Cache 一致性问题的情况：

- 当多个设备都允许访问主存时，比如 I/O 设备读取内存的数据，但是 Cache 中的副本已经被修改，那么读取的内容就是无效的
- 当多个 CPU 各自带有 Cache 而共享内存的时候，某个CPU修改了自身Cache中的内容，则对应的主存单元和其他CPU中对应的内容都变为无效

写操作由两种情况：要写的内容在 Cache（写命中）和 要写的内容不在 Cache（写不命中）

对于写命中，两种处理方式：

- 通写：同时写 Cache 和 主存。可以使用写缓冲来加速
- 一次性写：只写cache不写主存，缺失时一次写回，每行有个修改位，但控制可能很复杂

对于写不命中，两种处理方式：

- 写分配：先更新主存块内容，然后将更新后的主存块装入Cache，写完后将主存块读入Cache（因为空间局部性原理）
- 非写分配：直接写主存单元，不把主存块装入到Cache